

Two's complement

How computers store negative numbers, and why the ordinary adder still works

A byte is eight switches. It can hold 256 different patterns and not one of them is a minus sign. Yet every CPU adds, subtracts and compares negative numbers all day long. This tutorial explains the convention that makes that possible, **two's complement**, from first principles, with worked examples and a set of practice problems. Everything is shown in 8 bits because the patterns are short; the rules are identical at 16, 32 and 64 bits.

1. Bits have no sign
2. Place value, quickly
3. Two ideas that almost work
4. Two's complement: the top bit is negative
5. Writing a negative number
6. Why the arithmetic works: counting modulo 256
7. Overflow: when the true answer does not fit
8. The lopsided range
9. Changing the width: sign extension and truncation
10. Reading hex directly
11. Comparing signed and unsigned
12. Shifts
13. Practice problems, with answers
14. Cheat sheet

1. Bits have no sign

Write 5 in a byte and you get 0000 0101. Now try to write -5. There is nowhere to put the sign: a bit is a 0 or a 1, nothing else. So a negative number has to be *some pattern of bits* together with an agreement about how to read it. The machine does not know what a pattern means; it only knows how to add patterns together. A good convention is one where that adder, built for ordinary counting, produces the right answers for negative numbers too, without any extra circuitry. Two's complement is that convention, and it is the one every modern processor uses.

2. Place value, quickly

In an unsigned byte each bit has a weight, a power of two, counting from the right: 1, 2, 4, 8, 16, 32, 64, 128. The value is the sum of the weights whose bit is 1.

bit	7	6	5	4	3	2	1	0
weight	128	64	32	16	8	4	2	1
example	1	0	1	1	0	1	1	0

The example row is 1011 0110 = 128 + 32 + 16 + 4 + 2 = **182**.

Patterns get long, so they are written in **hex**: one hex digit stands for exactly four bits (a nibble), 0 to 9 and then A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. Group the bits in fours and translate each group: 1011 0110 is B6. The grouping in fours used throughout this document is there to make that translation easy.

nibble	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

3. Two ideas that almost work

Sign and magnitude

The obvious idea: use the top bit as the sign and the other seven as the size. 0000 0101 is +5 and 1000 0101 is -5. It reads naturally, but it fails the test that matters. Add +5 and -5 with an ordinary binary adder and you get 1000 0100, which reads as -10. The adder would need special circuits that look at the signs and sometimes subtract instead. It also has two zeros, 0000 0000 and 1000 0000, which every comparison then has to treat as equal.

Ones' complement

A better idea: to negate, flip every bit. +5 is 0000 0101, so -5 is 1111 1010. Now 5 + (-5) gives 1111 1111, which is the second zero of this system, and the adder is only right if a carry out of the top is fed back in at the bottom, the so-called end-around carry. Close, but still two zeros and still an adjustment.

Two's complement fixes both problems with one change: after flipping the bits, add one. Every negative number moves along by one, the two zeros collapse into one, and the plain adder becomes exactly right.

4. Two's complement: the top bit is negative

The whole convention fits in one sentence: **the top bit is worth -128 instead of $+128$** . All the other weights stay as they were.

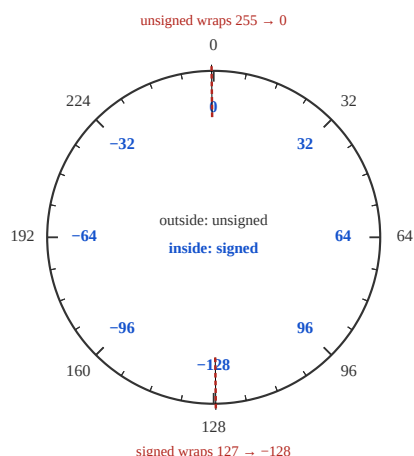
bit	7	6	5	4	3	2	1	0
weight	-128	64	32	16	8	4	2	1

To read a pattern, add up the weights exactly as before, remembering that the top one is negative.

pattern	hex	calculation	value
0000 0101	05	$4 + 1$	5
0111 1111	7F	$64 + 32 + 16 + 8 + 4 + 2 + 1$	127
1000 0000	80	-128	-128
1000 0001	81	$-128 + 1$	-127
1111 1011	FB	$-128 + 64 + 32 + 16 + 8 + 2 + 1$	-5
1111 1111	FF	$-128 + 127$	-1

Three things follow immediately from the negative weight:

- A pattern is negative exactly when its top bit is 1, because -128 outweighs everything the other seven bits can add (at most 127).
- The range is -128 to 127. The most negative value is the top bit alone; the most positive is everything but the top bit.
- -1 is all ones: the seven positive weights add to 127, and $127 - 128 = -1$. That is true at every width.



The 256 byte patterns laid around a circle. Unsigned reading counts all the way round; signed reading uses the same patterns but calls the top half -128 up to -1 .

The circle shows the key fact: unsigned and signed are *the same patterns*. Only the labels differ. The patterns from 00 to 7F mean the same thing either way; the patterns from 80 to FF are 128 to 255 unsigned and -128 to -1 signed. Each negative value is its unsigned reading minus 256.

pattern	unsigned	signed	note
0000 0000	0	0	one zero, the same in both readings
0000 0001	1	1	
0111 1111	127	127	largest positive signed value
1000 0000	128	-128	top bit set: the readings part company
1000 0001	129	-127	$129 - 256$
1111 1110	254	-2	$254 - 256$
1111 1111	255	-1	$255 - 256$

5. Writing a negative number

Reading is one direction; you also need to produce the pattern for a given negative number. There are three equivalent ways. Learn the first, use whichever you like.

Method 1: invert, then add one

Write the positive number, flip every bit, add 1.

```
5 = 0000 0101
flip = 1111 1010
+ 1 = 1111 1011 = FB = -5

1 = 0000 0001      100 = 0110 0100
flip = 1111 1110    flip = 1001 1011
+ 1 = 1111 1111 = -1  + 1 = 1001 1100 = 9C = -100
```

The same recipe turns a negative number back into a positive one: flip 1111 1011 to get 0000 0100, add 1, and 5 is back. Negating twice returns the original (with one exception, covered in section 8).

Why it works. A bit and its flip are always 0 and 1 in some order, so they add to 1 in every column. Therefore $x + \text{flip}(x)$ is all ones, 1111 1111, which reads as -1 . Rearranged: $\text{flip}(x) = -x - 1$. The flip on its own is one short of the negation, and the extra 1 makes up the difference: $\text{flip}(x) + 1 = -x$.

So the flip is *not* the negative. Flipping 5 gives 1111 1010 = -6 . Adding 1 gives -5 .

Method 2: subtract from 256

In 8 bits, $-x$ has the same pattern as $256 - x$. For -5 , compute $256 - 5 = 251$ and write 251 in binary: 1111 1011, the same pattern as before. In hex this is a one-line subtraction: $0x100 - 0x05 = 0xFB$. For 16 bits use 65536 ($0x10000$), for 32 bits 2^{32} , and in general 2^n . Section 6 explains why this is the same thing as method 1.

Method 3: the right-to-left shortcut

Reading from the right, copy the bits up to and including the first 1, then flip everything to the left of it.

```
100 = 0110 0100      rightmost 1 is bit 2
      01100 | 100      copy "100", flip "01100"
      10011 | 100
      = 1001 1100 = 9C = -100
```

This is method 1 done in one pass: the trailing zeros and the first 1 are exactly the bits that the "add one" carries through and restores.

6. Why the arithmetic works: counting modulo 256

A byte cannot hold 256 or anything larger. When an addition carries out of the top bit, that carry has nowhere to go and is simply dropped. Dropping a carry out of bit 7 subtracts exactly 256 from the true result, so a byte does arithmetic **modulo 256**: $250 + 10 = 260$, and the byte keeps $260 - 256 = 4$.

Now put the two facts together. The pattern for -5 is the pattern for 251. Adding 251 is, modulo 256, the same as subtracting 5. So an adder that only knows about unsigned numbers automatically subtracts, which is to say it does signed arithmetic, with no changes at all.

```
7 - 3 as 7 + (-3):    0000 0111      7
                     + 1111 1101      + 253 (the pattern for -3)
                     -----
                     1 0000 0100      = 260; drop the carry → 4 ✓

3 - 7 as 3 + (-7):    0000 0011      3
                     + 1111 1001      + 249 (the pattern for -7)
                     -----
                     1111 1100      = 252, which reads as -4 ✓

5 + (-5):             0000 0101
                     + 1111 1011
                     -----
                     1 0000 0000      drop the carry → 0 ✓
```

In the first and third sums a carry fell off the end; in the second none did. Either way the eight bits that remain are the right answer. The hardware never needs to know whether you meant the patterns as signed or unsigned. That is the point of the whole design: one adder, one subtractor, one multiplier for the low bits, and two ways of reading the result.

7. Overflow: when the true answer does not fit

Modulo arithmetic has a cost. If the true answer lies outside -128 to 127 , the eight bits you get are the right answer modulo 256 but the wrong answer as a signed number. This is **signed overflow**.

```
127 + 1:      0111 1111 + 0000 0001 = 1000 0000 = -128 (true answer 128 does not fit)
-128 - 1:     1000 0000 - 0000 0001 = 0111 1111 = 127  (true answer -129 does not fit)
100 + 50:     0110 0100 + 0011 0010 = 1001 0110 = -106 (true answer 150 does not fit)
```

Notice that nothing fell off the end in these three sums. Overflow is not about losing a carry; it is about the result landing on the wrong side of the sign boundary in the circle of section 4. The rule for spotting it is simple:

Overflow rule. Adding two numbers of the same sign can overflow, and it has overflowed exactly when the result has the opposite sign. Adding numbers of opposite signs can never overflow, because the true answer lies between them. (Subtraction follows by negating the second operand.)

The unsigned reading has its own, separate notion of "did not fit": a carry out of the top bit means the unsigned result exceeded 255. The two events are independent, which is easy to see from four small sums:

sum	8-bit result	carry out of bit 7?	unsigned reading	signed reading
5 + 3	0000 1000	no	8, correct	8, correct
255 + 1	0000 0000	yes	0, wrapped	$-1 + 1 = 0$, correct
100 + 50	1001 0110	no	150, correct	-106, overflow
200 + 100	0010 1100	yes	44, wrapped	$-56 + 100 = 44$, correct

In C, unsigned arithmetic is defined to wrap; signed overflow is undefined behaviour, which is why compilers are allowed to assume it never happens.

8. The lopsided range

There are 256 patterns. One of them is zero, so 255 are left for everything else, and 255 is odd: the positives and negatives cannot be equal in number. Two's complement gives the extra pattern to the negatives: -128 exists, $+128$ does not.

This has a famous consequence. Negate -128 by the recipe: flip 1000 0000 to get 0111 1111, add 1, and you are back at 1000 0000. The negation of -128 is -128 . In C, `abs(INT_MIN)` and `INT_MIN / -1` both overflow for this reason, and `-x` is not always positive for negative `x`. Every width has the same edge case: -32768 in 16 bits, $-2\,147\,483\,648$ in 32 bits.

9. Changing the width: sign extension and truncation

Widening

To make a signed number wider, copy its sign bit into all the new bits. This is **sign extension**.

```
-5  8-bit  1111 1011          = FB
    16-bit 1111 1111 1111 1011 = FFFB
    32-bit 1111 1111 1111 1111 1111 1111 1011 = FFFF FFFB

+5  8-bit  0000 0101          = 05
    16-bit 0000 0000 0000 0101 = 0005
```

Check the 16-bit case by reading it: the top bit is worth -32768 , and the rest add to 32763, so `FFFB` = -5 . Padding with zeros instead, called **zero extension**, is right for unsigned numbers and wrong for negative signed ones: `00FB` is 251, not -5 . This is the classic conversion bug. In C, widening a signed `int8_t` sign-extends and widening an unsigned `uint8_t` zero-extends; processors have separate load instructions for the two cases, for example `movsx` and `movzx` on x86, or `lb` and `lbu` on RISC-V.

Narrowing

To make a number narrower, keep the low bits and drop the rest. `FFFB` narrowed to 8 bits is `FB`, still -5 . `0100` (256) narrowed to 8 bits is `00`: the value is gone. Narrowing is safe only when the value fits the smaller range, which is the same as saying that every dropped bit is a copy of the new sign bit.

10. Reading hex directly

With a little practice you never need the binary.

- A signed value is negative when its first hex digit is 8 to F, because that is where the top bit lives. 7F is positive, 80 is negative; 7FFF is positive, 8000 is negative.
- To read a negative byte, subtract 256 from its unsigned value: FB = 251 - 256 = -5; 9C = 156 - 256 = -100. For 16 bits subtract 65536: FFFB = 65531 - 65536 = -5. For 32 bits subtract 2^{32} .
- To negate in hex, subtract every digit from F and then add 1: 25 → DA → DB, so -37 is DB. This is method 1 done a nibble at a time.

width	most negative	most positive	-1
8 bits	-128 80	127 7F	FF
16 bits	-32 768 8000	32 767 7FFF	FFFF
32 bits	-2 147 483 648 8000 0000	2 147 483 647 7FFF FFFF	FFFF FFFF
64 bits	-9 223 372 036 854 775 808 8000 0000 0000 0000	9 223 372 036 854 775 807 7FFF FFFF FFFF FFFF	FFFF FFFF FFFF FFFF

11. Comparing signed and unsigned

Because the two readings disagree about the top half of the patterns, they disagree about order too. 80 is larger than 7F unsigned (128 against 127) and smaller signed (-128 against 127). Processors therefore provide two families of comparison: "below" and "above" for unsigned, "less" and "greater" for signed, and the compiler picks one from the types in your program.

Mixing the readings in one expression is a well-known trap. In C the comparison `-1 < 1u` is false, because -1 is converted to unsigned, becomes FFFF FFFF, and that is the largest unsigned value there is.

12. Shifts

Shifting left one place doubles a number, and the top bit falls off. It can overflow just like addition: 0100 0000 (64) shifted left is 1000 0000, which reads as -128.

Shifting right halves a number, and the question is what to feed in at the top. A **logical** shift feeds in 0, which is right for unsigned values. An **arithmetic** shift feeds in a copy of the sign bit, which keeps negative numbers negative: 1111 1010 (-6) becomes 1111 1101 (-3), and 1111 1011 (-5) becomes 1111 1101 as well, so -5 halved rounds down to -3, toward minus infinity rather than toward zero. C leaves the right shift of a negative value implementation-defined; in practice compilers produce the arithmetic shift for signed types and the logical one for unsigned.

13. Practice problems

Work in 8 bits unless the problem says otherwise. Answers follow.

1. Read 1110 0011 as an unsigned number and as a signed number.
2. Write -20 as an 8-bit pattern, in binary and in hex.
3. Read 9C as a signed byte, then negate it by flipping and adding one and read the result.
4. 7F + 02: give the 8-bit result and say whether it is a signed overflow.
5. F0 + 20: give the 8-bit result, say whether a carry fell off the top, and say whether it is a signed overflow.
6. Sign-extend B4 to 16 bits, then zero-extend it instead. Read both results as signed 16-bit numbers.
7. Which is larger, C8 or 32, as unsigned bytes and as signed bytes?
8. In 16 bits, what is the pattern for -1000? Give it in hex.
9. Read 8000 as a signed 16-bit number. What happens when you negate it?
10. Explain why 8000 - 0001 = 7FFF is an overflow in the signed reading.
11. Read the 16-bit pattern 1111 1111 1111 1011, then narrow it to 8 bits. Does the value survive, and how could you have known without reading it?
12. Read the 32-bit pattern FFFF FF9C as a signed number.

Answers

1. Unsigned: $128 + 64 + 32 + 2 + 1 = 227$. Signed: the top bit is set, so $227 - 256 = -29$.
2. 20 is 0001 0100; flip to 1110 1011; add 1 to get 1110 1100 = EC. Check: $236 - 256 = -20$.
3. $9C = 156 - 256 = -100$. Flipping 1001 1100 gives 0110 0011; adding 1 gives 0110 0100 = 64 = 100.
4. 81, which reads as -127. Yes: $127 + 2 = 129$ does not fit, and two positives gave a negative.
5. $10 = 16$, and a carry fell off the top ($0xF0 + 0x20 = 0x110$). Not a signed overflow: the operands have opposite signs (-16 and 32) and $-16 + 32 = 16$ is exactly what the result says.
6. Sign-extended: $FFB4 = -76$ (the same value as the byte, since $B4 = 180 - 256 = -76$). Zero-extended: $00B4 = 180$, a different number.
7. Unsigned: C8 (200 against 50). Signed: 32 (50 against -56).
8. 1000 is 03E8; flip to FC17; add 1 to get FC18. Check: $65536 - 1000 = 64536 = FC18$.
9. -32768, the most negative 16-bit value. Negating it gives 8000 again: flip to 7FFF, add 1, and you are back where you started.
0. The true answer is $-32768 - 1 = -32769$, which is below the range, and the pattern 7FFF reads as +32767. A negative minus a positive produced a positive: overflow.
1. It is -5 ($65531 - 65536$). Narrowed to FB it is still -5. The value survives because every dropped bit was a 1, a copy of the new sign bit, so nothing but redundant sign copies was thrown away.
2. The first digit is F, so it is negative: $4\ 294\ 967\ 196 - 4\ 294\ 967\ 296 = -100$. Faster: the low byte is $9C = -100$ and the upper bytes are all sign copies.

14. Cheat sheet

Reading	Add the bit weights; the top bit is worth -2^{n-1} . A pattern is negative exactly when its top bit is 1.
Range	-2^{n-1} to $2^{n-1} - 1$. For 8 bits: -128 to 127. One more negative than positive.
Landmarks	-1 is all ones. The most negative value is 1 followed by zeros. Zero is unique.
Negating	Flip every bit and add 1; equivalently, subtract from 2^n ; equivalently, copy up to the first 1 from the right and flip the rest. Negating the most negative value gives it back.
Flip alone	$\text{flip}(x) = -x - 1$, because $x + \text{flip}(x)$ is all ones, which is -1.
Arithmetic	The adder does not know about signs. Everything is modulo 2^n , and $-x$ is the same pattern as $2^n - x$, so adding the pattern for $-x$ subtracts x .
Overflow	Only when adding numbers of the same sign; it has happened when the result has the other sign. Independent of whether a carry fell off the top.
Widening	Copy the sign bit into the new bits (sign extension). Zero extension is for unsigned values only.
Narrowing	Keep the low bits. Safe only when every dropped bit equals the new sign bit.
Hex	First digit 8 to F means negative. Negative value = unsigned value - 2^n . Negate by subtracting each digit from F and adding 1.
Order	Signed and unsigned readings order the top half of the patterns oppositely; never compare the two readings in one expression.

Practise the same ideas interactively with the Programmer Calculator at snowch.github.io/programmer-calculator, whose exercise groups on reading and operating on two's complement follow this tutorial.